

HDOC Day-8 Activity

Question 1:

Write a C program to find the radius, curved surface area, and volume of a hemisphere when its total surface area is given.

Step 1: Understanding the Problem

Input: Total surface area (TSA) of the hemisphere.

Output: Radius r , curved surface area (CSA), and volume V .

Formula: $TSA = 3\pi r^2$, so $r = \sqrt{(TSA / 3\pi)}$; $CSA = 2\pi r^2$; $V = (2/3)\pi r^3$.

Step 2: Standard Test Case Table

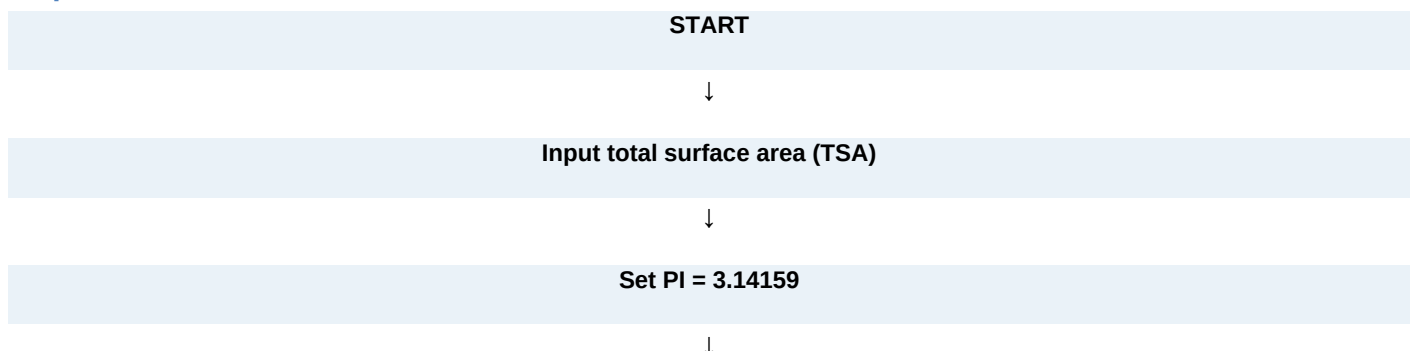
Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	TSA=942.477	$r=10.00$, $CSA=628.32$, $V=2094.39$	$r=10.00$, $CSA=628.32$, $V=2094.39$	Pass
2	TSA=2356.194	$r=15.81$, $CSA=1570.80$, $V=8278.82$	$r=15.81$, $CSA=1570.80$, $V=8278.82$	Pass
3	TSA=3769.911	$r=20.00$, $CSA=2513.27$, $V=16755.17$	$r=20.00$, $CSA=2513.27$, $V=16755.17$	Pass

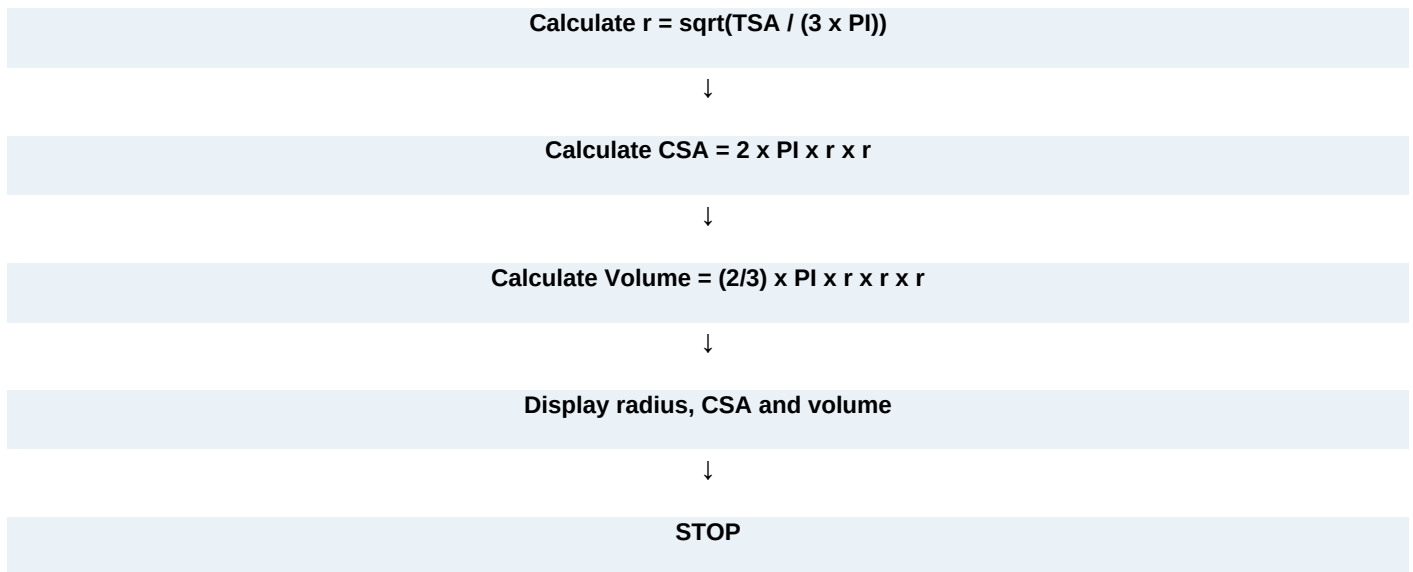
Step 3: Algorithm and Evaluation

1. Start.
2. Declare tsa, radius, csa, volume and PI.
3. Read total surface area from the user.
4. Calculate radius = $\sqrt{tsa / (3 * PI)}$.
5. Calculate csa = $2 * PI * radius * radius$.
6. Calculate volume = $(2.0 / 3.0) * PI * radius * radius * radius$.
7. Display radius, curved surface area and volume.
8. Stop.

Evaluation: The algorithm first obtains the radius from the total surface area and then uses that radius to calculate curved surface area and volume. It gives the expected result for all three test cases.

Step 4: Flowchart and Evaluation





Evaluation: The flowchart follows the same sequence as the algorithm and produces the expected results for the prepared test cases.

Step 5: C Programming

```
#include <stdio.h>
#include <math.h>

int main()
{
    double tsa, radius, csa, volume;
    const double PI = 3.14159;

    printf("Enter total surface area: ");
    scanf("%lf", &tsa);

    radius = sqrt(tsa / (3 * PI));
    csa = 2 * PI * radius * radius;
    volume = (2.0 / 3.0) * PI * radius * radius * radius;

    printf("Radius = %.2f\n", radius);
    printf("Curved Surface Area = %.2f\n", csa);
    printf("Volume = %.2f\n", volume);
    return 0;
}
```

Step 6: Compilation, Execution and Evaluation

The program was compiled and executed in Kali Linux and evaluated against the three test cases prepared in Step 2. All three produced the expected output.

Step 7: Kali Linux Compilation and Execution

```
(kali@kali)-[~]
$ nano hemisphere.c
(kali@kali)-[~]
$ gcc hemisphere.c -o hemisphere -lm
(kali@kali)-[~]
$ ./hemisphere
Enter total surface area: 942.477
Radius = 10.00
Curved Surface Area = 628.32
Volume = 2094.39
```

Test Case 2:

```
(kali@kali)-[~]  
$ ./hemisphere  
Enter total surface area: 2356.194  
Radius = 15.81  
Curved Surface Area = 1570.80  
Volume = 8278.82
```

Test Case 3:

```
(kali@kali)-[~]  
$ ./hemisphere  
Enter total surface area: 3769.911  
Radius = 20.00  
Curved Surface Area = 2513.27  
Volume = 16755.17
```

Question 2:

Write a C program to find the new coordinates of a point after rotation by 180 degrees about the origin.

Step 1: Understanding the Problem

Input: Original coordinates x and y.

Output: New coordinates (x', y') after a 180-degree rotation about the origin.

Formula: $(x, y) \rightarrow (-x, -y)$.

Step 2: Standard Test Case Table

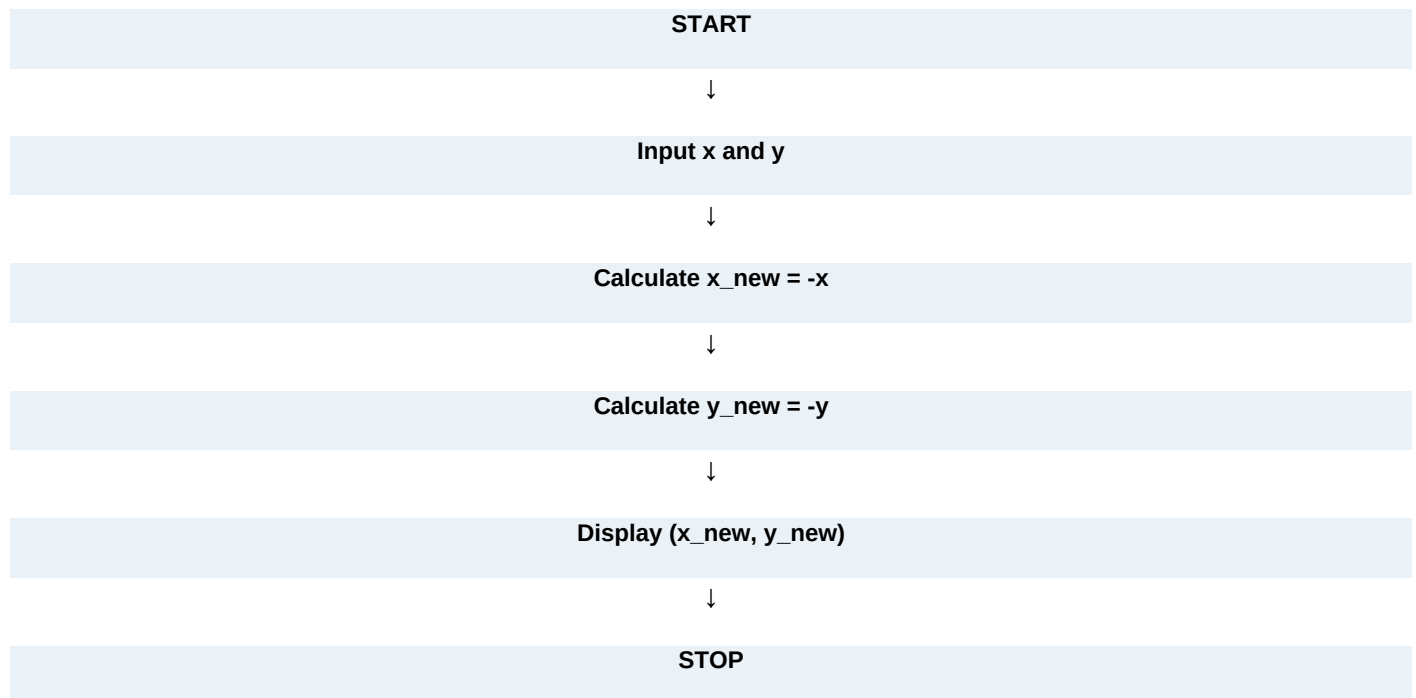
Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	(3, 4)	(-3.00, -4.00)	(-3.00, -4.00)	Pass
2	(-5, 2)	(5.00, -2.00)	(5.00, -2.00)	Pass
3	(0, -7)	(0.00, 7.00)	(0.00, 7.00)	Pass

Step 3: Algorithm and Evaluation

1. Start.
2. Declare x, y, x_new and y_new.
3. Read x and y from the user.
4. Calculate $x_new = -x$.
5. Calculate $y_new = -y$.
6. Display the new coordinates (x_new, y_new).
7. Stop.

Evaluation: A 180-degree rotation about the origin changes the sign of both coordinates. Therefore the algorithm correctly transforms every test point.

Step 4: Flowchart and Evaluation



Evaluation: Negating both coordinates is exactly the transformation for a 180-degree rotation about the origin, so the flowchart matches all prepared test cases.

Step 5: C Programming

```
#include <stdio.h>

int main()
{
    float x, y, x_new, y_new;

    printf("Enter x coordinate: ");
    scanf("%f", &x);
    printf("Enter y coordinate: ");
    scanf("%f", &y);

    x_new = -x;
    y_new = -y;

    printf("New coordinates = (%.2f, %.2f)\n", x_new, y_new);
    return 0;
}
```

Step 6: Compilation, Execution and Evaluation

The program was compiled and executed in Kali Linux and checked against the three test cases prepared in Step 2. All three test cases passed.

Step 7: Kali Linux Compilation and Execution

```
(kali@kali)-[~]
$ nano rotation.c
(kali@kali)-[~]
$ gcc rotation.c -o rotation
(kali@kali)-[~]
$ ./rotation
Enter x coordinate: 3
```

```
Enter y coordinate: 4  
New coordinates = (-3.00, -4.00)
```

Test Case 2:

```
(kali@kali)-[~]  
$ ./rotation  
Enter x coordinate: -5  
Enter y coordinate: 2  
New coordinates = (5.00, -2.00)
```

Test Case 3:

```
(kali@kali)-[~]  
$ ./rotation  
Enter x coordinate: 0  
Enter y coordinate: -7  
New coordinates = (0.00, 7.00)
```